57. BullMQ + Worker Pods + KEDA: Node.js Jobs That Autoscale

Builds on **#49 (Redis)**, **#50 (Docker)**, **#53 (Kubernetes)**, **#55 (KubeRay)** and **#56 (async and threads)**.

> **In one line:** the API puts jobs in a **Redis queue (BullMQ)**, separate **worker pods** take jobs and process them, and **KEDA** adds more worker pods when the queue is long and removes them (down to zero) when it's empty. This is the **Node.js version of KubeRay**.

Our real example: send 1,000 welcome emails 📧

The admin clicks "Send welcome email to all students". Sending one email takes about **2 seconds**. Doing 1,000 emails inside the API request would take **33 minutes**, and the request would time out. Instead:

1. The API **adds 1,000 jobs** to a queue and answers immediately: `{ queued: 1000 }` ⚡
2. **Worker pods** send the emails in the background.
3. **KEDA** sees 1,000 waiting jobs and scales the workers from **0** → **10 pods**. When everything is done, it goes back to **0** 💰

PART 1: The 3 pieces

1. Bank token example 🏦

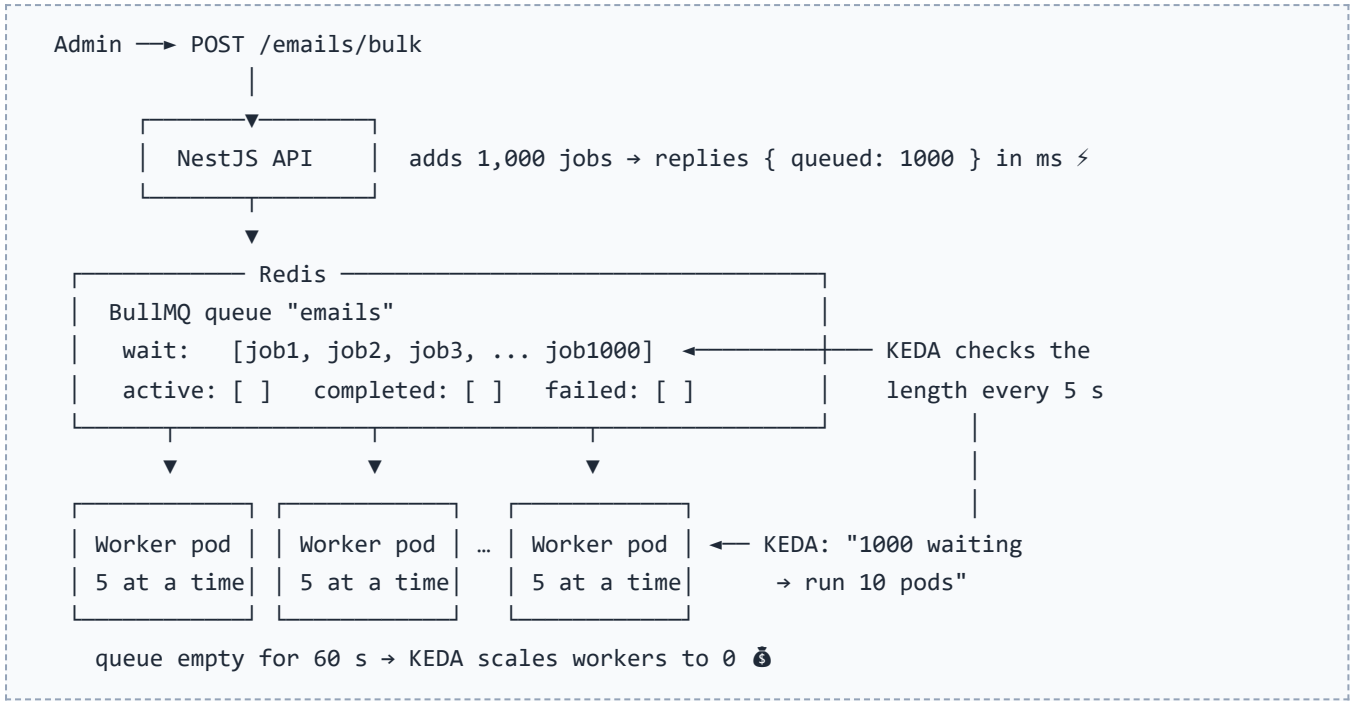
> **Customers** = API requests ("send this email").
> **Token machine + waiting hall** = **BullMQ queue in Redis**. Each customer gets a token (a job) and waits in line.
> **Counters / cashiers** = **worker pods**. Each calls the next token and serves it.
> **Branch manager** = **KEDA**. They watch the hall: "50 people waiting? Open 5 more counters!" "Hall empty? Close all counters and save salary."

Piece	What it is	Job in our example
BullMQ	A Node.js library for **job queues**, stored in **Redis**	Holds the 1,000 "send email" jobs, with retries and status tracking
Producer (the API)	Code that **adds** jobs	NestJS `POST /emails/bulk`
Worker (consumer)	Code that **takes and processes** jobs	Sends each email
Worker pods	The worker app running in Kubernetes pods (#53)	0 to 10 copies of the email worker
KEDA	**K**ubernetes **E**vent-**D**riven **A**utoscaler	Scales worker pods based on the **queue length**

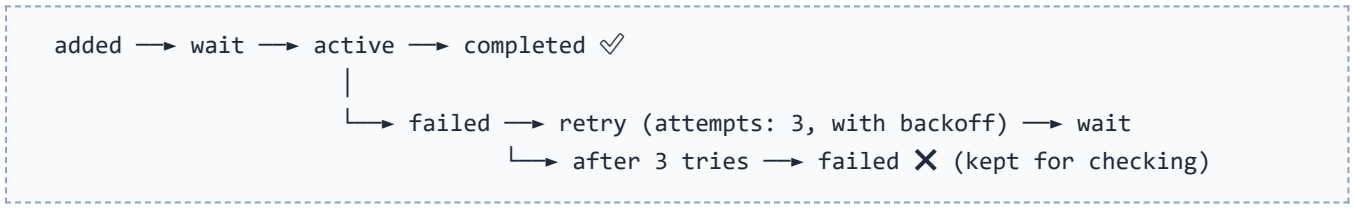
A little history

- **Bull** was a popular Redis queue library for Node. **BullMQ** is its modern **TypeScript rewrite** by the same author (Manuel Astudillo / Taskforce.sh). NestJS has official support: `@nestjs/bullmq`.
- **KEDA** was created by **Microsoft and Red Hat** in 2019. It's now a graduated **CNCF** project, like Kubernetes itself.
- **Why KEDA?** Normal Kubernetes autoscaling (HPA) only looks at **CPU/memory** and can't go to **zero**. A worker waiting for jobs uses almost no CPU, so HPA wouldn't know there are 1,000 jobs waiting. KEDA scales on **events**: queue length, Kafka messages, cron schedules, and more.

2. How it works



A job's life



PART 2: Build it locally (no Kubernetes yet)

3. Start Redis

```
docker run -d --name redis -p 6379:6379 redis:7
```

4. Install

```
npm install @nestjs/bullmq bullmq
```

Project structure: one project, two entry points

```
src/
├── main.ts                  ← API entry   (node dist/main.js)
├── worker.main.ts           ← WORKER entry (node dist/worker.main.js)
```

```
├── app.module.ts              ← API module (adds jobs, does NOT process them)
├── worker.module.ts           ← Worker module (processes jobs)
└── emails/
    ├── emails.controller.ts   ← POST /emails/bulk, GET /emails/:id
    ├── emails.service.ts      ← producer
    └── email.processor.ts     ← consumer (worker)
```

> The API and the worker share code and the same Docker image, but run as **separate processes**, so you can scale them **separately**: 2 API pods and 0–10 worker pods.

5. The API side (producer)

app.module.ts

```ts
import { Module } from '@nestjs/common';
import { BullModule } from '@nestjs/bullmq';
import { EmailsController } from './emails/emails.controller';
import { EmailsService } from './emails/emails.service';

@Module({
  imports: [
    BullModule.forRoot({
      connection: {
        host: process.env.REDIS_HOST ?? 'localhost',
        port: Number(process.env.REDIS_PORT ?? 6379),
      },
    }),
    BullModule.registerQueue({ name: 'emails' }),
  ],
  controllers: [EmailsController],
  providers: [EmailsService],          // ← no EmailProcessor here
})
export class AppModule {}
```

emails/emails.service.ts

```ts
import { Injectable, NotFoundException } from '@nestjs/common';
import { InjectQueue } from '@nestjs/bullmq';
import { Queue } from 'bullmq';

export interface EmailJob { to: string; name: string; }

@Injectable()
export class EmailsService {
  constructor(@InjectQueue('emails') private readonly queue: Queue<EmailJob>) {}

  // Pretend these are students from Postgres
  async sendWelcomeToAll(count: number) {
    const jobs = Array.from({ length: count }, (_, i) => ({
      name: 'welcome',
```

```
      data: { to: `student${i + 1}@school.com`, name: `Student ${i + 1}` },
      opts: {
        attempts: 3,                                    // retry up to 3 times
        backoff: { type: 'exponential', delay: 2000 },  // wait 2s, 4s, 8s between tries
        removeOnComplete: 1000,                         // keep last 1000 finished jobs
        removeOnFail: 5000,                             // keep failed jobs for checking
      },
    }));
    await this.queue.addBulk(jobs);
    return { queued: count };
  }

  async status(id: string) {
    const job = await this.queue.getJob(id);
    if (!job) throw new NotFoundException('Job not found');
    return {
      id: job.id,
      state: await job.getState(),       // waiting | active | completed | failed | delayed
      progress: job.progress,
      result: job.returnvalue,
      error: job.failedReason,
      attempts: job.attemptsMade,
    };
  }

  async counts() {
    return this.queue.getJobCounts('waiting', 'active', 'completed', 'failed');
  }
}
```

emails/emails.controller.ts

```
import { Body, Controller, Get, Param, ParseIntPipe, Post } from '@nestjs/common';
import { EmailsService } from './emails.service';

@Controller('emails')
export class EmailsController {
  constructor(private readonly emails: EmailsService) {}

  @Post('bulk')
  bulk(@Body('count', ParseIntPipe) count: number) {
    return this.emails.sendWelcomeToAll(count);     // returns in milliseconds ⚡
  }

  @Get('stats')
  stats() {
    return this.emails.counts();
  }

  @Get(':id')
  status(@Param('id') id: string) {
    return this.emails.status(id);
```

```
  }
}
```

Put `@Get('stats')` **before** `@Get(':id')`; otherwise "stats" is treated as an id.

6. The worker side (consumer)

emails/email.processor.ts

```
import { Logger } from '@nestjs/common';
import { Processor, WorkerHost, OnWorkerEvent } from '@nestjs/bullmq';
import { Job } from 'bullmq';
import { hostname } from 'os';
import { EmailJob } from './emails.service';

@Processor('emails', { concurrency: 5 })      // each pod handles 5 jobs at a time
export class EmailProcessor extends WorkerHost {
  private readonly logger = new Logger(EmailProcessor.name);

  async process(job: Job<EmailJob>) {
    this.logger.log(`[${hostname()}] sending to ${job.data.to}`);
    await job.updateProgress(50);

    // 👉 real app: await this.mailer.send(...)
    await new Promise((r) => setTimeout(r, 2000));       // pretend: email takes 2 s

    await job.updateProgress(100);
    return { sentAt: new Date().toISOString(), by: hostname() };
  }

  @OnWorkerEvent('failed')
  onFailed(job: Job, err: Error) {
    this.logger.error(`Job ${job.id} failed (try ${job.attemptsMade}): ${err.message}`);
  }
}
```

worker.module.ts

```
import { Module } from '@nestjs/common';
import { BullModule } from '@nestjs/bullmq';
import { EmailProcessor } from './emails/email.processor';

@Module({
  imports: [
    BullModule.forRoot({
      connection: {
        host: process.env.REDIS_HOST ?? 'localhost',
        port: Number(process.env.REDIS_PORT ?? 6379),
      },
    }),
    BullModule.registerQueue({ name: 'emails' }),
```

```
  ],
  providers: [EmailProcessor],         // ← only the worker
})
export class WorkerModule {}
```

worker.main.ts: no HTTP server, just the worker

```
import { NestFactory } from '@nestjs/core';
import { WorkerModule } from './worker.module';

async function bootstrap() {
  const app = await NestFactory.createApplicationContext(WorkerModule);
  app.enableShutdownHooks();      // on SIGTERM: stop taking jobs, finish current ones
  console.log('📧 Email worker started');
}
bootstrap();
```

7. Run and test locally

```
# Terminal 1: API
npm run start:dev

# Terminal 2: worker #1
npx nest start --watch --entryFile worker.main

# Terminal 3: add 100 jobs
curl -X POST http://localhost:3000/emails/bulk -H "Content-Type: application/json" -d
'{"count":100}'
# → { "queued": 100 }   (instant)

curl http://localhost:3000/emails/stats
# → { "waiting": 95, "active": 5, "completed": 0, "failed": 0 }

curl http://localhost:3000/emails/1
# → { "id":"1", "state":"completed", "progress":100,
#     "result": { "sentAt":"...", "by":"LAPTOP-GOURI" }, ... }
```

Now start a 2nd worker in Terminal 4 (same command)

Both workers take jobs from the **same queue**, and no job is processed twice. BullMQ uses Redis to make sure each job goes to exactly one worker at a time.

Workers	Jobs at a time	100 jobs × 2 s each
1 worker (concurrency 5)	5	~40 s
2 workers	10	~20 s
10 workers	50	~4 s 🚀

> This is the whole idea: **more workers = faster**. Starting them by hand doesn't scale, so **Kubernetes + KEDA** does it automatically (Part 3).

PART 3: Run on Kubernetes with KEDA autoscaling

8. One Docker image for both

```
# Dockerfile
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build
# default = API; the worker Deployment overrides the command
CMD ["node", "dist/main.js"]
```

```
docker build -t school-app:1.0 .
```

On Docker Desktop's Kubernetes, local images work directly (use `imagePullPolicy: IfNotPresent`). On a cloud cluster, push the image to Docker Hub or ECR first.

9. Install KEDA (once per cluster)

```
helm repo add kedacore https://kedacore.github.io/charts
helm repo update
helm install keda kedacore/keda --namespace keda --create-namespace

kubectl get pods -n keda          # keda-operator ... Running ✓
```

10. Kubernetes YAML: k8s/app.yaml

```
# ---------- Redis (OK for learning; use managed Redis in production) ----------
apiVersion: apps/v1
kind: Deployment
metadata: { name: redis }
spec:
  replicas: 1
  selector: { matchLabels: { app: redis } }
  template:
    metadata: { labels: { app: redis } }
    spec:
      containers:
```

```
        - name: redis
          image: redis:7
          ports: [{ containerPort: 6379 }]
---
apiVersion: v1
kind: Service
metadata: { name: redis }
spec:
  selector: { app: redis }
  ports: [{ port: 6379 }]
---
# ---------- API (always 2 copies) ----------
apiVersion: apps/v1
kind: Deployment
metadata: { name: api }
spec:
  replicas: 2
  selector: { matchLabels: { app: api } }
  template:
    metadata: { labels: { app: api } }
    spec:
      containers:
        - name: api
          image: school-app:1.0
          imagePullPolicy: IfNotPresent
          command: ["node", "dist/main.js"]
          env: [{ name: REDIS_HOST, value: redis }]
          ports: [{ containerPort: 3000 }]
---
apiVersion: v1
kind: Service
metadata: { name: api }
spec:
  type: LoadBalancer
  selector: { app: api }
  ports: [{ port: 3000, targetPort: 3000 }]
---
# ---------- WORKER (KEDA controls the number of copies) ----------
apiVersion: apps/v1
kind: Deployment
metadata: { name: email-worker }
spec:
  replicas: 0                               # KEDA will change this
  selector: { matchLabels: { app: email-worker } }
  template:
    metadata: { labels: { app: email-worker } }
    spec:
      terminationGracePeriodSeconds: 60    # time to finish current jobs on shutdown
      containers:
        - name: worker
          image: school-app:1.0
          imagePullPolicy: IfNotPresent
          command: ["node", "dist/worker.main.js"]     # ← same image, worker mode
          env: [{ name: REDIS_HOST, value: redis }]
          resources:
```

```
            requests: { cpu: "100m", memory: "128Mi" }
            limits:   { cpu: "500m", memory: "256Mi" }
```

11. The KEDA part: k8s/keda.yaml ⭐

```
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: email-worker-scaler
spec:
  scaleTargetRef:
    name: email-worker            # the Deployment to scale
  pollingInterval: 5              # check the queue every 5 seconds
  cooldownPeriod: 60              # wait 60 s of "empty" before scaling to 0
  minReplicaCount: 0              # 0 pods when no jobs 💰
  maxReplicaCount: 10             # never more than 10 pods
  triggers:
    - type: redis
      metadata:
        address: redis.default.svc.cluster.local:6379   # full DNS: KEDA runs in another
namespace
        listName: bull:emails:wait                      # BullMQ's waiting list in Redis
        listLength: "20"                                # target: 1 pod per 20 waiting jobs
```

Setting	Meaning
`listName: bull:emails:wait`	BullMQ stores waiting jobs in a Redis list called `bull:<queue>:wait`. KEDA reads its length (`LLEN`).
`listLength: "20"`	Pods wanted ≈ waiting jobs ÷ 20 (capped at `maxReplicaCount`)
`minReplicaCount: 0`	Scale to zero: no cost when idle
`cooldownPeriod: 60`	Don't jump to 0 the moment the list is empty; wait 60 s
`pollingInterval: 5`	How often KEDA checks Redis

> **Check the key name yourself:** run `kubectl exec -it deploy/redis -- redis-cli KEYS "bull:*"`. If you set a custom BullMQ `prefix`, the list name changes to match.

12. Deploy and watch it scale 👀

```
kubectl apply -f k8s/app.yaml
kubectl apply -f k8s/keda.yaml

kubectl get scaledobject          # READY: True, ACTIVE: False (no jobs yet)
kubectl get pods                  # redis, api x2, NO worker pods (0) ✅
```

```
# terminal 2: watch pods live
kubectl get pods -l app=email-worker -w

# terminal 1: add 1,000 jobs
kubectl port-forward svc/api 3000:3000
curl -X POST http://localhost:3000/emails/bulk -H "Content-Type: application/json" -d
'{"count":1000}'
```

What you'll see

Time	Waiting jobs	Worker pods	What happened
0 s	0	0	Idle: costs nothing 💰
1 s	1000	0	API added 1,000 jobs (replied instantly)
~5 s	1000	1	KEDA: "activate!" 0 → 1
~20 s	~950	**10**	KEDA (through HPA): 1000 ÷ 20 = 50 → capped at 10
~60 s	~0	10	10 pods × 5 jobs at a time = 50 parallel emails
after cooldown	0	**0**	Queue empty for 60 s → scaled back to zero

The time it takes to scale up depends on how fast pods start. Watch logs from all workers at once: `kubectl logs -l app=email-worker -f --prefix`. Each line shows which pod (hostname) sent which email.

PART 4: Make it production-safe

13. Graceful shutdown (very important with autoscaling)

When KEDA scales **down**, Kubernetes sends **SIGTERM** to a worker pod. If the pod just dies, its in-progress emails are interrupted.

What to set	Why
`app.enableShutdownHooks()` in `worker.main.ts`	Nest closes the BullMQ worker: it stops taking new jobs and lets current jobs finish
`terminationGracePeriodSeconds: 60`	Gives the pod time to finish. Must be **longer than your slowest job**.
`attempts: 3` + `backoff`	If a pod is killed anyway, BullMQ detects the **stalled** job and another worker retries it
Make jobs **idempotent**	Running the same job twice must be safe (e.g. check "already sent?" in the DB before sending)

14. Useful extras

Feature	Code	Use
Delay	`queue.add('x', data, { delay: 60_000 })`	Send a reminder in 1 minute
Priority	`{ priority: 1 }` (lower = sooner)	Password-reset emails before newsletters
Repeat / cron	`{ repeat: { pattern: '0 2 * * *' } }`	Nightly report at 2 AM
Rate limit	`@Processor('emails', { limiter: { max: 10, duration: 1000 } })`	Max 10 emails/second (email provider limits)
Dashboard	**Bull Board** (`@bull-board/nestjs`)	A web UI showing waiting, active, failed and retry jobs
CPU-heavy jobs	Piscina inside the worker (#56)	Each pod also uses all of its cores

Priority and delayed jobs are stored in other Redis keys, not in `bull:emails:wait`, so KEDA's list count won't see them. If you use them heavily, add more triggers, or keep `minReplicaCount: 1`.

15. Common problems

Problem	Why	Fix
Worker pods never start	KEDA can't reach Redis	Use the full DNS name `redis.default.svc.cluster.local:6379`, then check `kubectl describe scaledobject`
KEDA always sees 0 jobs	Wrong `listName` or prefix	Check with `redis-cli KEYS "bull:*"`
Jobs added but never processed (locally)	The worker isn't running, or uses a different Redis/queue name	Start `worker.main`, and use the same queue name everywhere
The API also processes jobs	`EmailProcessor` is in `AppModule`	Keep processors only in `WorkerModule`
Emails sent twice	Job retried after a crash	Make jobs idempotent (check before sending)
Redis memory keeps growing	Completed jobs are kept forever	`removeOnComplete` / `removeOnFail`
Jobs cut off when scaling down	No graceful shutdown	`enableShutdownHooks()` + a longer grace period

16. Compared with KubeRay and threads

	Piscina (#56)	BullMQ + KEDA (this)	KubeRay (#55)
Language	Node	Node	Python
Machines	1	Many	Many
Hands out work	Thread pool	**Redis queue**	Ray head
Autoscaling	✗	✓ KEDA (on queue length)	✓ Ray autoscaler
Scale to zero	—	✓	✓
Retries, delay, priority	DIY	✓ built-in	✓ task retries
Best for	CPU work in one app	**Background jobs**: emails, reports, images, imports	ML / AI / big data
Laptop-friendly?	✓	✓ (workers ~100 MB each)	⚠ heavy (GBs)

17. Quick summary

Question	Answer
BullMQ?	A Node.js job queue library stored in **Redis**
Producer?	The NestJS API: `queue.add()` / `addBulk()`, which replies instantly
Worker?	A `@Processor` class in a **separate process/pod** (`worker.main.ts`)
Worker pods?	The same Docker image, run with `node dist/worker.main.js`
KEDA?	A Kubernetes autoscaler driven by **events** (here: the length of `bull:emails:wait`)
Why not HPA?	HPA scales on CPU and can't scale to zero; KEDA scales on queue length and can
Key KEDA settings	`listName`, `listLength`, `min/maxReplicaCount`, `cooldownPeriod`
Safety	Retries + backoff, graceful shutdown, idempotent jobs, `removeOnComplete`
Learning path	Run locally with 1 → 2 workers → Docker → Kubernetes → add KEDA